

Plano de Desenvolvimento de Software

Portal SENAI

Lucas Lopes
Computer Engineer

2026

Resumo

Este documento apresenta a especificação detalhada de requisitos e o plano cronológico para o desenvolvimento de um ecossistema web utilizando a arquitetura formada por **Spring Boot** no back-end, **React (Vite) + Bootstrap** no front-end e **PostgreSQL** como banco de dados. O projeto adota uma estratégia pedagógica baseada em um portal corporativo de cursos e notícias do SENAI, voltado ao portfólio pessoal do orientador e à formação prática dos estudantes.

1 Arquitetura de Referência do Sistema

O ecossistema de software foi modelado de forma a garantir desacoplamento, segurança baseada em estados sem sessão (*stateless*) e responsividade para o usuário final. A topologia segue o fluxo:

[React + Vite + Bootstrap] $\xrightarrow{\text{Ações / JWT}}$ [API REST - Spring Boot] $\xrightarrow{\text{JPA / Hibernate}}$ [PostgreSQL]

2 Cronograma de Requisitos Automatizados por Fase

Fase 1: Modelagem e Banco de Dados (PostgreSQL)

Antes da codificação, realiza-se a estruturação do esquema relacional que suportará as regras de negócio da plataforma.

- **RFDB-01:** Criação física das instâncias dos bancos de dados relacionais isolados no PostgreSQL: `db_senahub` (ambiente de aula/exemplo) e `db_portal` (ambiente oficial do projeto).
- **RFDB-02 (Tabela de Usuários):** Criação da tabela `tb_usuario` contendo os atributos primários: identificador único incremental (*Primary Key*), nome completo, email corporativo/pessoal (*Unique Index*), hash criptografado da senha, nível de acesso (*Role/Enum*: `ROLE_ADMIN`, `ROLE_ALUNO`) e o indicador booleano `pode_postar_noticias`.
- **RFDB-03 (Tabela de Conteúdo Principal):** Criação da tabela `tb_curso` contendo os atributos: ID, nome, descrição técnica, carga horária, modalidade e total de vagas.
- **RFDB-04 (Tabela de Notícias):** Criação da tabela `tb_noticia` mapeando ID, título, corpo do texto, timestamp de criação, string de referência do recurso multimídia (*URL da imagem de capa*) e relacionamento de chave estrangeira (*Foreign Key*) vinculada à tabela `tb_usuario`.

Fase 2: Back-End Base (Spring Boot)

Construção da arquitetura lógica inicial do servidor REST, expondo recursos e persistência de dados sem restrições de segurança primárias para validação de fluxo.

- **RFBE-01 (Setup Inicial):** Inicialização do projeto Java via *Spring Initializr* configurando o gerenciador de dependências e os módulos: `Spring Web`, `Spring Data JPA`, `PostgreSQL Driver`, `Lombok` e `Validation`.
- **RFBE-02 (CRUD de Notícias):** Implementação do padrão arquitetural de camadas (*Entities, Repositories, Services e Controllers*) para a entidade Notícias. Disponibilização obrigatória do endpoint público `GET /api/noticias` configurado nativamente com paginação dinâmica (*Pageable*).
- **RFBE-03 (CRUD de Cursos):** Exposição dos caminhos `GET/POST /api/cursos`.
- **RFBE-04 (Gerenciamento de Arquivos):** Desenvolvimento de um componente de serviço para interceptar requisições do tipo binário (*MultipartFile*) responsável por salvar as mídias em diretório físico ou serviço de armazenamento em nuvem.

Fase 3: Segurança e Controle de Acesso (Spring Security + JWT)

Implementação de regras restritivas e barreira de segurança stateless por meio do protocolo de assinatura digital JWT.

- **RFBE-05 (Filtros de Segurança):** Acoplamento das dependências **Spring Security** e biblioteca **java-jwt**. Escrita da classe **SecurityFilter** herdando de **OncePerRequestFilter** para validação da assinatura do token no cabeçalho **Authorization**.
- **RFBE-06 (Autenticação):** Exposição de rotas públicas livres de verificação para o fluxo de registro e login: **POST /api/auth/register** e **POST /api/auth/login**, retornando um *JSON Web Token* ativo na resposta.
- **RFBE-07 (Controle Baseado em Perfis - RBAC):** Restrição estrita de endpoints de modificação técnica (Cursos) apenas para perfis autenticados específicos. Criação do endpoint especializado **PATCH /api/usuarios/id/permissoes-noticia**, acessível estritamente pelo perfil **ROLE_ADMIN**.
- **RFBE-08 (Validação de Regra Customizada):** Implementação de lógica condicional na camada de serviço de publicação: rejeitar a operação se o usuário não contiver privilégio de administrador ou se o booleano **pode_postar_noticias** for computado como falso (**false**).

Fase 4: Front-End Base & Integração (React + Vite)

Desenvolvimento da camada de visualização em Single Page Application (SPA), estilização em Bootstrap e comunicação assíncrona orientada a serviços.

- **RFFE-01 (Setup & Tematização):** Geração do andaime do projeto via *Vite*. Instalação do Bootstrap. Customização das variáveis globais através do pré-processador Sass (`.scss`) para sobreposição da paleta visual nativa:
 - **Identidade Visual SENAI:** Predomínio de tons em Azul Industrial (`#005691`), alinhado à identidade institucional do SENAI.
- **RFFE-02 (Gerenciamento de Rotas e Sessão):** Configuração das rotas via `react-router-dom`. Desenvolvimento do `AuthContext` utilizando a Context API para prover centralização das credenciais, tratamento e persistência do token JWT no cache `localStorage`.
- **RFFE-03 (Páginas Públicas):** Construção da *Landing Page* utilizando os seletores de grid e colunas do Bootstrap, consumindo dinamicamente a API `GET /api/noticias` via biblioteca cliente `Axios`.
- **RFFE-04 (Formulários de Login):** Construção das telas de captura para autenticação, utilizando pseudoclasses e classes nativas do Bootstrap de validação de formulários (`.was-validated`).
- **RFFE-05 (Formulário Avançado de Notícias):** Tela de escrita de notícias estruturada para envio de dados compostos (`FormData`), contendo campos textuais e binários (arquivos de imagem), anexando o token de segurança nos cabeçalhos HTTP (`Bearer Token`).

Fase 5: Dashboards

Refinamento da experiência do usuário (UX) através da exibição gráfica de indicadores analíticos corporativos.

- **RFFE-06 (Painel Administrativo):** Rota protegida do administrador contendo listagem de dados tabulares. Implementação do componente switch de controle do Bootstrap (`form-check form-switch`) acoplado à requisição assíncrona imediata via método `PATCH`.
- **RFFE-07 (Interface Reativa Condicional):** Componentização visual do menu de navegação lateral (*Sidebar*). O recurso “Escrever Notícia” deve ter sua renderização no DOM vinculada diretamente à leitura das permissões do usuário logado.
- **RFFE-08 (Plotagem de Gráficos Analíticos):** Acoplamento e configuração de componentes analíticos visuais utilizando a biblioteca `Recharts` ou `Chart.js`:
 - **Dashboard SENAI:** Exibição em formato de gráfico de pizza/rosca mapeando os cursos com maior taxa de procura e interesse.

Fase 6: Deploy e Homologação (Portfólio Pronto)

Finalização do ecossistema, separação de variáveis críticas do código e disponibilização pública em nuvem de produção.

- **RFDE-01 (Isolamento de Credenciais):** Migração de propriedades estáticas para injeção via variáveis de ambiente no arquivo `application.properties` ou `application.yml` do Spring Boot (chaves simétricas de criptografia, string de conexão de produção).
- **RFDE-02 (Deploy do Back-End):** Provisionamento e implantação contínua da API Spring Boot e da base PostgreSQL utilizando infraestrutura PaaS (*Platform as a Service*) como Render, Railway ou AWS.
- **RFDE-03 (Deploy do Front-End):** Geração do pacote estático otimizado (`npm run build`) da aplicação React e hospedagem em plataformas de alta performance de borda, como Vercel ou Netlify.
- **RFDE-04 (Documentação e Portfólio):** Confecção do arquivo estrutural `README.md` no repositório do GitHub. O arquivo deve conter capturas visuais da aplicação executando, especificação técnica da arquitetura construída, instruções completas para execução e teste em ambientes locais, e os hiperlinks funcionais da aplicação em produção.